ARM controllers:

Introduction:

The British computer manufacturer Acorn Computers first developed the Acorn RISC Machine architecture (ARM) in the 1980s to use in its personal computers. The official *Acorn RISC Machine* project started in October 1983. They chose VLSI Technology as the *silicon partner*, as they were a source of ROMs and custom chips for Acorn. Wilson and Furber led the design.

https://www.mepits.com/tutorial/194/arm/arm.

- **Some of the advantages of ARM processors are listed below:**

- They are cheap as compared to other processors.
- They are designed with such features that it consumes less power.
- The devices with ARM processor can have a much better battery life than other processors.
- They perform one operation at a time and thus work faster.
- The availability and applications support has also made the users choose ARM processors.

- ARM processors are a particular type of processors which are made by the ARM holdings PLC. This processor is also known as **Reduced Instruction Set Computing (RISC)** in which simple central processing units produce a high performance quality for the users. It has 32 bits RISC load architecture. Nowadays ARM processor are widely used in a number of electronic products such as mobile phones, tablets, multi-media players and many more electrical devices. The direct administration of the memory is not possible in this design and is handled by the use of registers. They are also used in personal digital assistants, digital media and music layers, hand-held gaming systems, and hard drives of the computer.

ARM processors vs. Intel processors: When you purchase a processor you will get to know that some of the designs use Intel Atom processors while the other uses ARM architecture. These both processors are designed in a way for low-power operation. They are created to give the mobile a long battery life. But they are both based on different ideology. The **ARM** structure is made very simple and is intended to keep the energy wastage to the minimum. Intel processors are made of complicated designs to match the compatibility of the company‘s desktop and laptop CPU’s. ARM processors are being used in the portable devices from a very long time but Intel has just entered to this area. Intel is believed to have a serious competition with the ARM technology. The power chips of ARM processors are relatively low in price, so if Intel will have to reconsider their pricing strategies to compete with it. The entry of ARM processors and their wide usage has forced Intel to increase their efficiency.

- Simulation is the process of using a simulation software (simulator) to verify the functional correctness of a digital design that is modeled using a HDL (hardware description language) like Verilog.

- Synthesis is a process in which a design behavior that is modeled using a HDL is translated into an implementation consisting of logic gates. This is done by a synthesis tool which is another software program.

Difference between 8051, AVR, PIC, ARM microcontrollers:

https://www.elprocus.com/difference-between-avr-arm-8051-and-pic-microcontroller/.

http://www.firmcodes.com/difference-arm-microcontrollers/.

8051 Microcontroller

8051 microcontroller is an 8-bit family of microcontroller is developed by the **Intel** in the year 1981. This is one of the popular families of microcontroller are being used all across the world. This microcontroller was moreover referred as “system on a chip” since it has 128 bytes of RAM, 4Kbytes of a ROM, 2 Timers, 1 Serial port, and 4 ports on a single chip. The CPU can also work for 8bits of data at a time since 8051 is an 8-bit processor. In case the data is bigger than 8 bits, then it has to be broken into parts so that the CPU can process easily. Most manufacturers contain put 4Kbytes of ROM even though the number of ROM can be exceeded up to 64 K bytes.

PIC Microcontroller

Peripheral Interface Controller (PIC) is microcontroller developed by a **Microchip**, PIC microcontroller , PIC microcontroller is fast and simple to implement program when we contrast other microcontrollers like 8051. The ease of programming and simple to interfacing with other peripherals PIC become successful microcontroller. We know that microcontroller is an integrated chip which is consists of RAM, ROM, CPU, TIMER and COUNTERS, PIC microcontroller is fast and simple to implement program when we contrast other microcontrollers like 8051. The ease of programming and simple to interfacing with other peripherals PIC become successful microcontroller. We know that microcontroller is an integrated chip which is consists of RAM, ROM, CPU, TIMER and COUNTERS. The PIC is a microcontroller which as well consists of RAM, ROM, CPU, timer, counter, ADC (analog to digital converters, PIC microcontroller is fast and simple to implement program when we contrast other microcontrollers like 8051. The ease of programming and simple to interfacing with other peripherals PIC become successful microcontroller. We know that microcontroller is an integrated chip which is consists of RAM, ROM, CPU, TIMER and COUNTERS. The PIC is a microcontroller which as well consists of RAM, ROM, CPU, timer, counter, ADC (analog to digital converters), DAC (digital to analog converter). PIC Microcontroller also support the protocols like CAN, SPI, UART for an interfacing with additional peripherals. PIC mostly used to

AVR Microcontroller

AVR microcontroller was developed in the year of 1996 by Atmel Corporation. The structural design of AVR was developed by the Alf-Egil Bogen and Vegard Wollan. AVR derives its name from its developers and stands for Alf-Egil Bogen Vegard Wollan RISC microcontroller, also known as Advanced Virtual RISC. The AT90S8515 was the initial microcontroller which was based on the AVR architecture, though the first microcontroller to hit the commercial market was AT90S1200 in the year 1997.

AVR Microcontrollers are Available in three Categories:

TinyAVR:- Less memory, small size, appropriate just for simpler applications.
MegaAVR:- These are the mainly popular ones having a good quantity of memory (up to 256 KB), higher number of inbuilt peripherals and appropriate for modest to complex applications.
XmegaAVR:- Used in commercial for complex applications, which need large program memory and high speed.

Main Difference between AVR, ARM, 8051 and PIC Microcontrollers

	8051	PIC	AVR	ARM
Bus width	8-bit for standard core	8/16/32-bit	8/32-bit	32-bit mostly also available in 64-bit
Communication Protocols	UART, USART,SPI,I2C	PIC, UART, USART, LIN, CAN, Ethernet, SPI, I2S	UART, USART, SPI, I2C, (special purpose AVR support CAN, USB, Ethernet)	UART, USART, LIN, I2C, SPI, CAN, USB, Ethernet, I2S, DSP, SAI (serial audio interface), IrDA
Speed	12 Clock/instruction cycle	4 Clock/instruction cycle	1 clock/ instruction cycle	1 clock/ instruction cycle
Memory	ROM, SRAM, FLASH	SRAM, FLASH	Flash, SRAM, EEPROM	Flash, SDRAM, EEPROM
ISA	CLSC	Some feature of RISC	RISC	RISC
Memory Architecture	Von Neumann architecture	Harvard architecture	Modified	Modified Harvard architecture
Power Consumption	Average	Low	Low	Low
Families	8051 variants	PIC16,PIC17, PIC18, PIC24, PIC32	Tiny, Atmega, Xmega, special purpose AVR	ARMv4,5,6,7 and series
Community	Vast	Very Good	Very Good	Vast

Manufacturer	NXP, Atmel, Silicon Labs, Dallas, Cyprus, Infineon, etc.	Microchip Average	Atmel	Apple, Nvidia, Qualcomm, Samsung Electronics, and TI etc.
Cost (as compared to features provide)	Very Low	Average	Average	Low
Other Feature	Known for its Standard	Cheap	Cheap, effective	High speed operation Vast
Popular Microcontrollers	AT89C51, P89v51, etc.	PIC18fXX8, PIC16f88X, PIC32MXX	Atmega8, 16, 32, Arduino Community	LPC2148, ARM Cortex-M0 to ARM Cortex-M7, etc.

ARM

- ARM: Advanced RISC Machines
- Most widely used 32- bit RISC instruction set architecture
- The relative simplicity makes it suitable for low power devices
- ARM7, ARM9, ARM11 and Cortex
- Approximately 90% of all embedded 32-bit RISC processors
- Used extensively in consumer electronics, including PDAs, mobile phones, digital media and music players, hand-held game consoles, calculators and computer peripherals such as hard drives and routers.

Product Code Description

- M: Multiplier
  ARM processor have hardware multiplier unit doing multiplication
- I: Embedded ICE Macrocel
  Hardware circuit used to generate trace information. Used in advance debugging.
- E: Enhanced Instruction Set
- J: Java Acceleration by Jazelle mode
  Hardware circuit used for running JAVA byte code
- F: Vector Floating point
  Hardware implementation of floating operations.
- S: Synthesizable Version
  The ARM architecture can be modified as it comes in terms of soft processor core.

ARM POWERED ™

ARM

- ARM has 3 instruction set states
  1. 32-bit **ARM** instruction set
  2. 16-bit **Thumb** instruction set
  3. 8- bit **Jazelle** instruction set
- ARM – 32 bit Load/Store architecture with every instruction being **conditional.**
- Thumb- 16 bit with only branch instructions being conditional and only half of the registers used
- Jazelle- Allows Java byte code to be directly executed in ARM architecture. Improves performance by 5x-10x

ARM POWERED ™

Example

- ARM7TDMI
  This is the ARM7 family processor which has T= Thumb instruction set, D= Debug Unit, M= MMU(Memory Management Unit), I= Embedded Trace core.
- ARM946E-S
  1. ARM9xx core
  2. Enhanced Instruction set
  3. Synthesizable

ARM POWERED ™

Main features of the ARM Instruction Set

* **All instructions are 32 bits long.**
* **Most instructions execute in a single cycle.**
* **Every instruction can be conditionally executed.**
* **A load/store architecture**
  - Data processing instructions act only on registers
    - Three operand format
    - Combined ALU and shifter for high speed bit manipulation
  - Specific memory access instructions with powerful auto-indexing addressing modes.
    - 32 bit and 8 bit data types and also 16 bit data types on ARM Architecture v4.
    - Flexible multiple register load and store instructions
* **Instruction set extension via coprocessors**

ARM POWERED ™

Processor (Operational) Modes

- **The ARM has seven operating modes:**
  - **User** : unprivileged mode under which most tasks run
  - **FIQ** : entered when a high priority (fast) interrupt is raised
  - **IRQ** : entered when a low priority (normal) interrupt is raised
  - **SVC** : (Supervisor) entered on reset and when a Software Interrupt instruction is executed
  - **Abort** : used to handle memory access violations
  - **Undef** : used to handle undefined instructions
  - **System** : privileged mode using the same registers as user mode

ARM POWERED ™

The Registers

- **ARM has 37 registers all of which are 32-bits long.**
  - 1 dedicated program counter
  - 1 dedicated current program status register
  - 5 dedicated saved program status registers
  - 30 general purpose registers

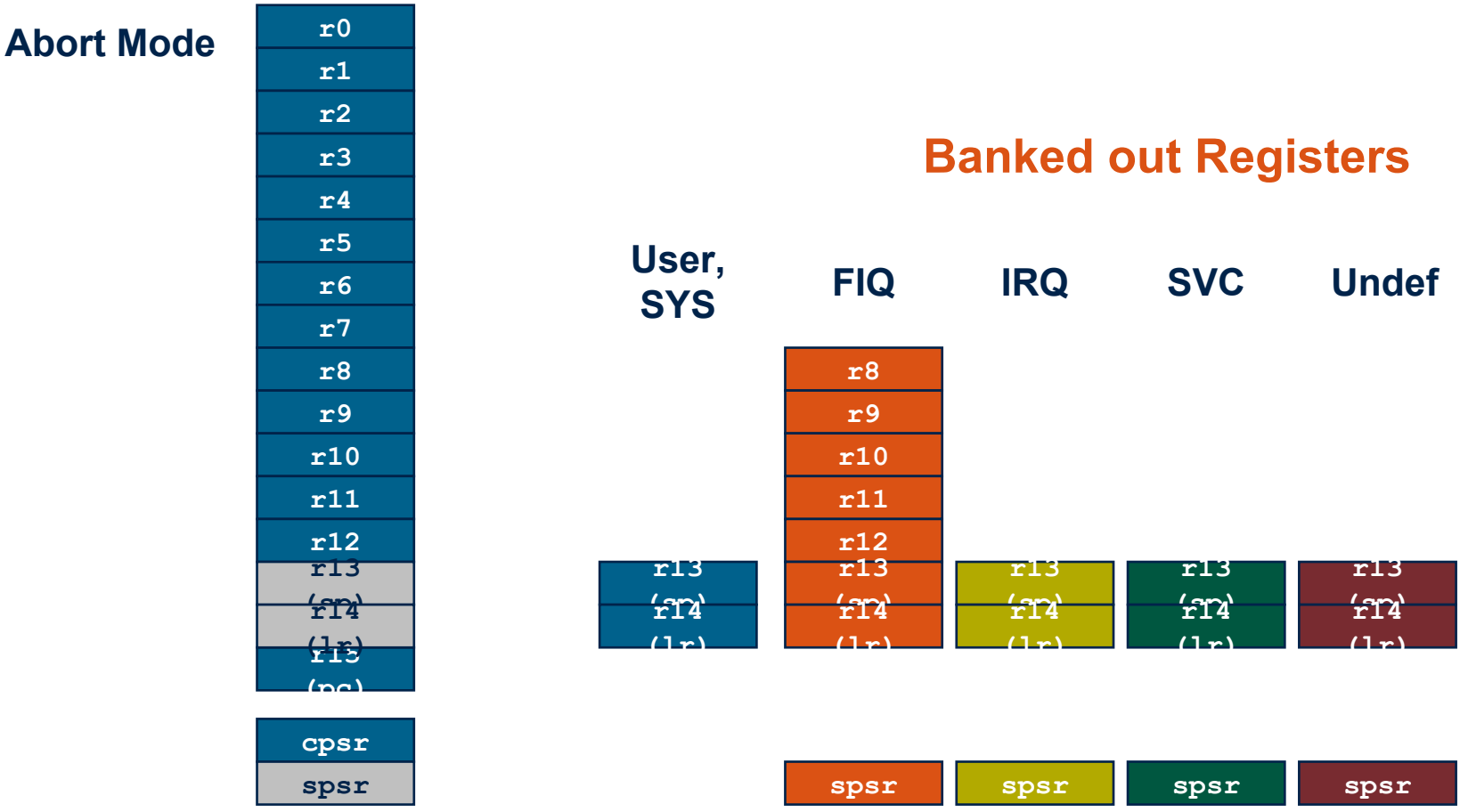
- **The current processor mode governs which of several banks is accessible. Each mode can access**
  - a particular set of r0-r12 registers
  - a particular r13 (the stack pointer, sp) and r14 (the link register, lr)
  - the program counter, r15 (pc)
  - the current program status register, cpsr

  Privileged modes (except System) can also access
  - a particular spsr (saved program status register)

ARM POWERED ™

The ARM Register Set

Current Visible Registers

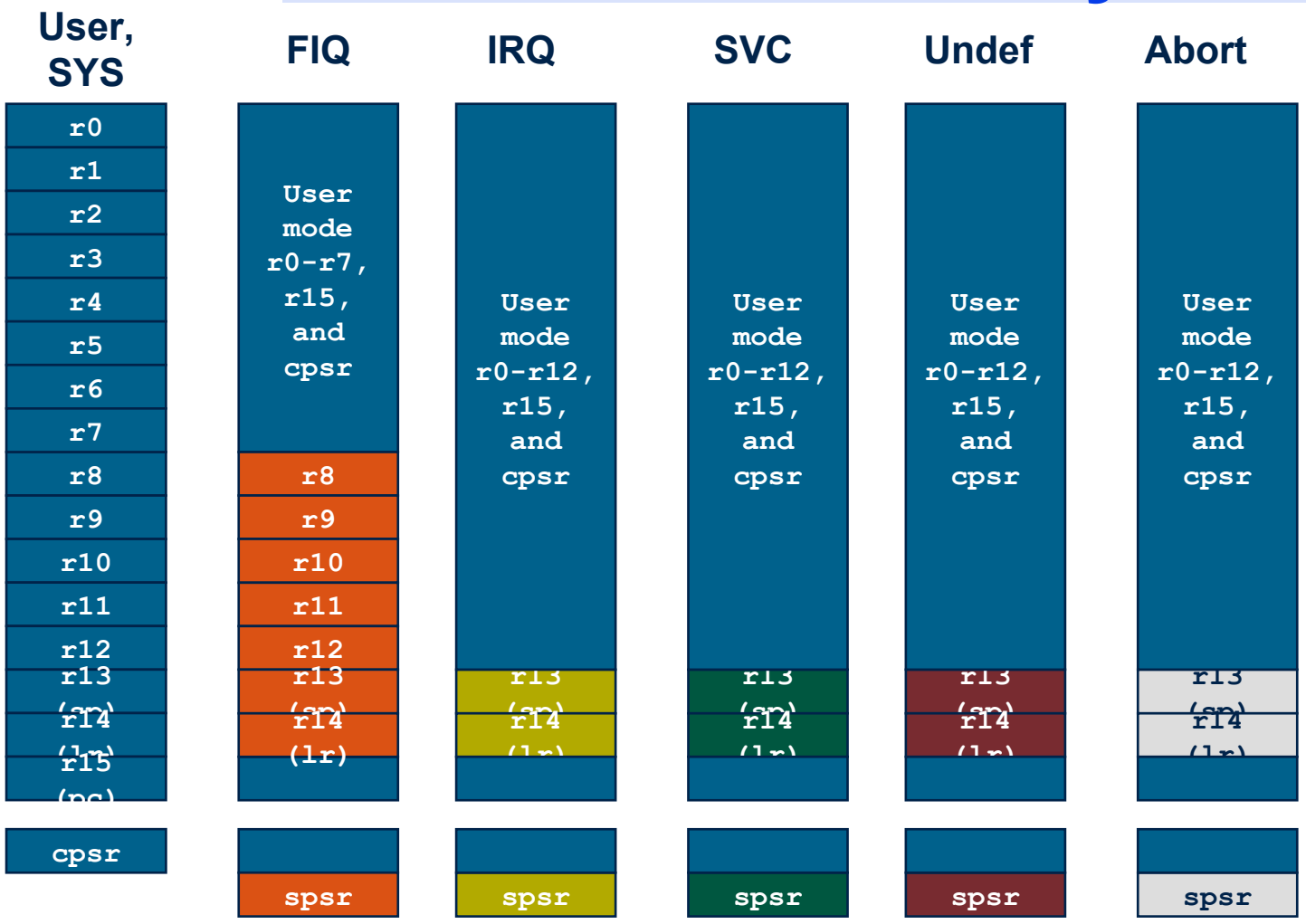


Special Registers

- **Special function registers:**
  - **PC** (R15): Program Counter. Any instruction with PC as its destination register is a program branch.
  - **LR** (R14): Link Register. Saves a copy of PC when executing the BL instruction (subroutine call) or when jumping to an exception or interrupt routine
    - It is copied back to PC on the return from those routines
  - **SP** (R13): Stack Pointer. There is **no stack** in the ARM architecture. Even so, R13 is usually reserved as a pointer for the program-managed stack
  - **CPSR** : Current Program Status Register. Holds the visible status register
  - **SPSR** : Saved Program Status Register. Holds a copy of the previous status register while executing exception or interrupt routines
    - It is copied back to CPSR on the return from the exception or interrupt
    - No SPSR available in User or System modes

ARM POWERED ™

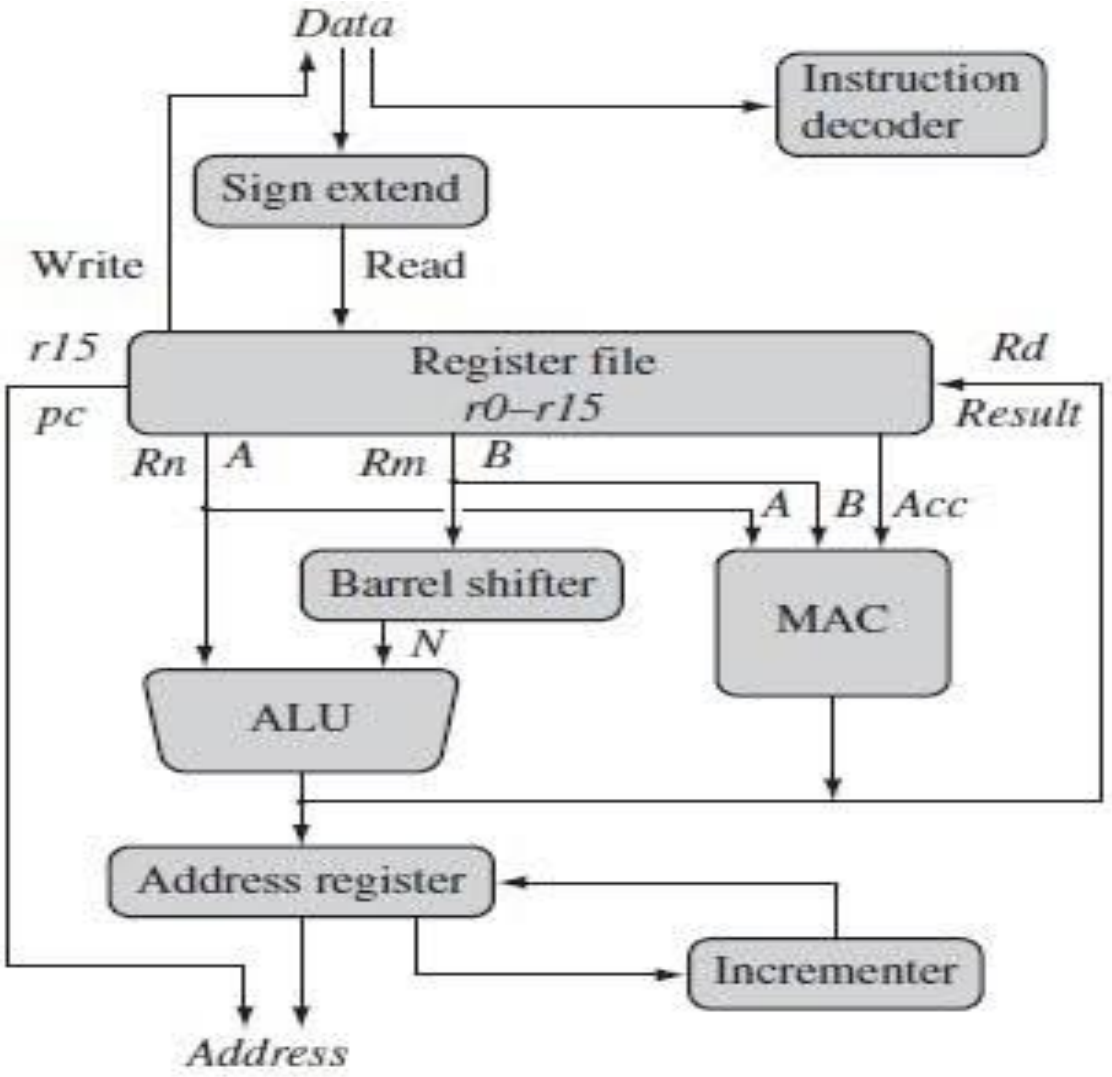
Register Organization Summary



Note: System mode uses the User mode register set

ARM POWERED ™

ARM Core Data Flow Model



Definition: When an instruction is decoded inside the ARM core and how a particular instruction is executed by interacting with the internal registers file and then send result out of the registers.

 Von Neumann Architecture Hence data coming through bus is either instruction or data (same memory).

 Instruction decoder translates instructions before they are executed.

- Load instruction: copy data from memory to register
- Store instruction: copy data from register to memory

 The Sign extend hardware converts signed 8-bit & 16-bit numbers to 32-bit values as they are read from memory & placed in a register (for signed values), fill zeros if unsigned.

- There are no data processing instructions that directly manipulate data in memory.
- Data items are placed in the register file – a storage bank made up of 32-bit registers.

 Source operands (Rn & Rm) are read from the register file using the internal buses A & B respectively & result Rd is written back.

- ARM instructions typically have two source registers Rn and Rm and one destination register Rd.

 The PC value is in the address register which is fed in to the incrementer, then the incremented value is copied back in to r15.

 It is also written in to address register to be used as the address for the next instruction fetch.

 ALU: (The Arithmetic & logic Unit) or MAC (multiply & accumulate Unit) takes the register values Rn & Rm from A & B buses & computers a result)

- ALU and MAC (Multiply-accumulate) unit takes the register values Rn and Rm from A and B buses and computes a result.
- Load and store instructions use the ALU to generate an address to be held in the address register and broadcast on the Address bus.

 Data processing instructions write the result in Rd directly to the register file.

● The register Rm can be alternatively pre-processed in the barrel shifter before it enters the ALU.

 Load & Store instruction use the ALU to generate on Address to be to be held in the address register & broadcast on the address bus.

● For load and store instructions the incrementer updates the address register before the core reads or writes the next register value from or to the next sequential memory location.

 Barrel shifter:

•One important feature of the is that register Rm alternatively can be pre processed in barrel in barrel shifter before it enters the ALU [left shift , right shift , rotated etc.

•Depending on the instruction Barrel Shifter may be used or it could be short circuit.

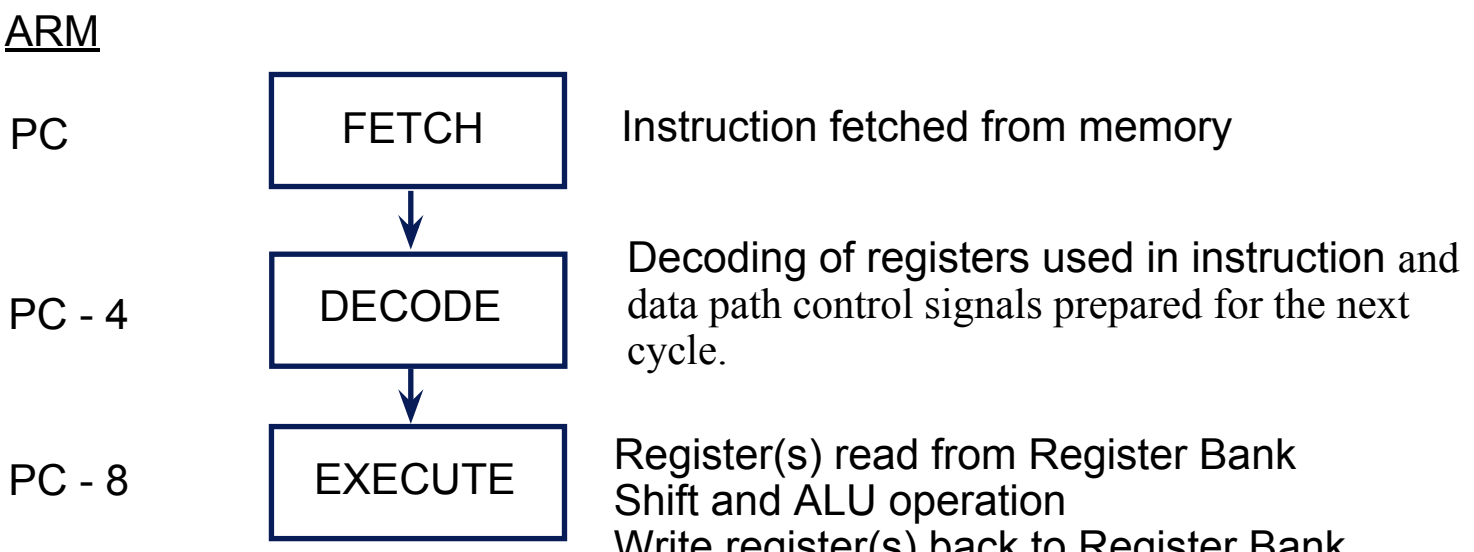
•Barrel shifter & ALU can calculate together a wide range of expression & address in the same cycle.

ARM Pipelining examples

- Fetch: Read Op-code from memory to internal Instruction Register
- Decode: Activate the appropriate control lines depending on Opcode
- Execute: Do the actual processing

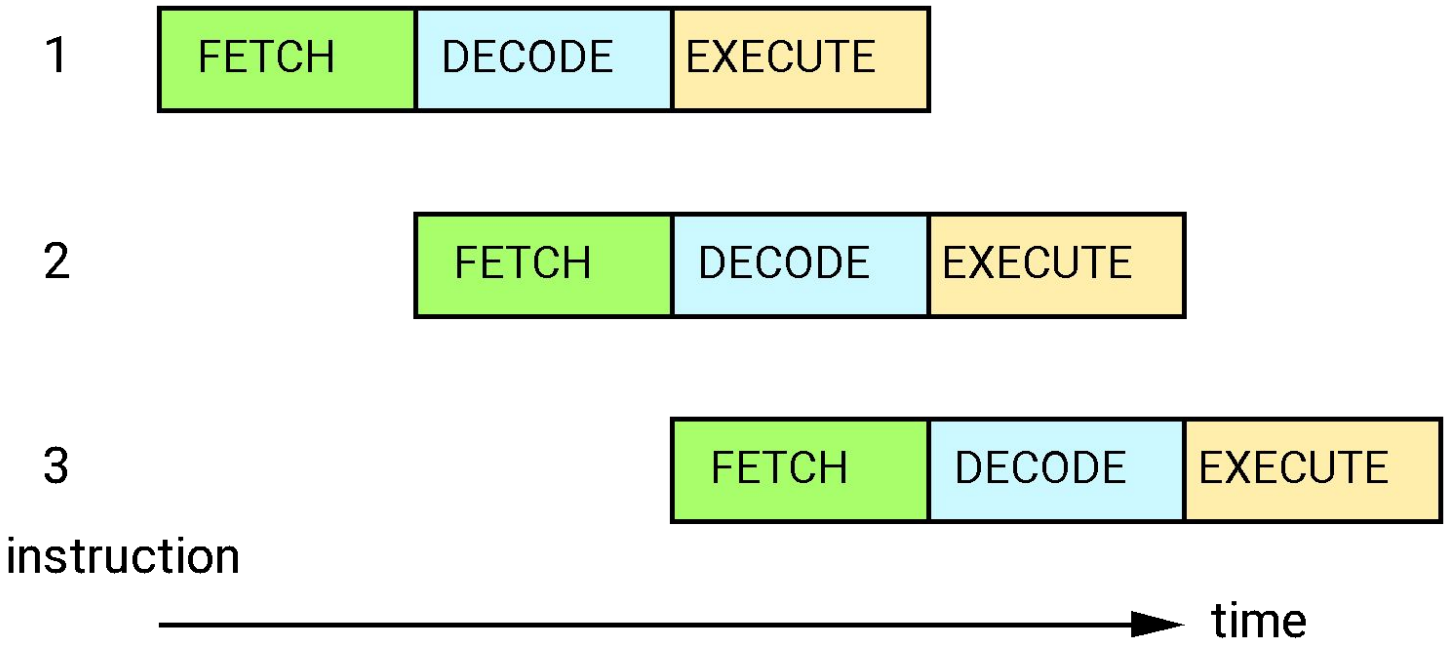
The Instruction Pipeline

* **The ARM uses a pipeline in order to increase the speed of the flow of instructions to the processor.**
  - Allows several operations to be undertaken simultaneously, rather than serially.



* **Rather than pointing to the instruction being executed, the PC points to the instruction being fetched.**

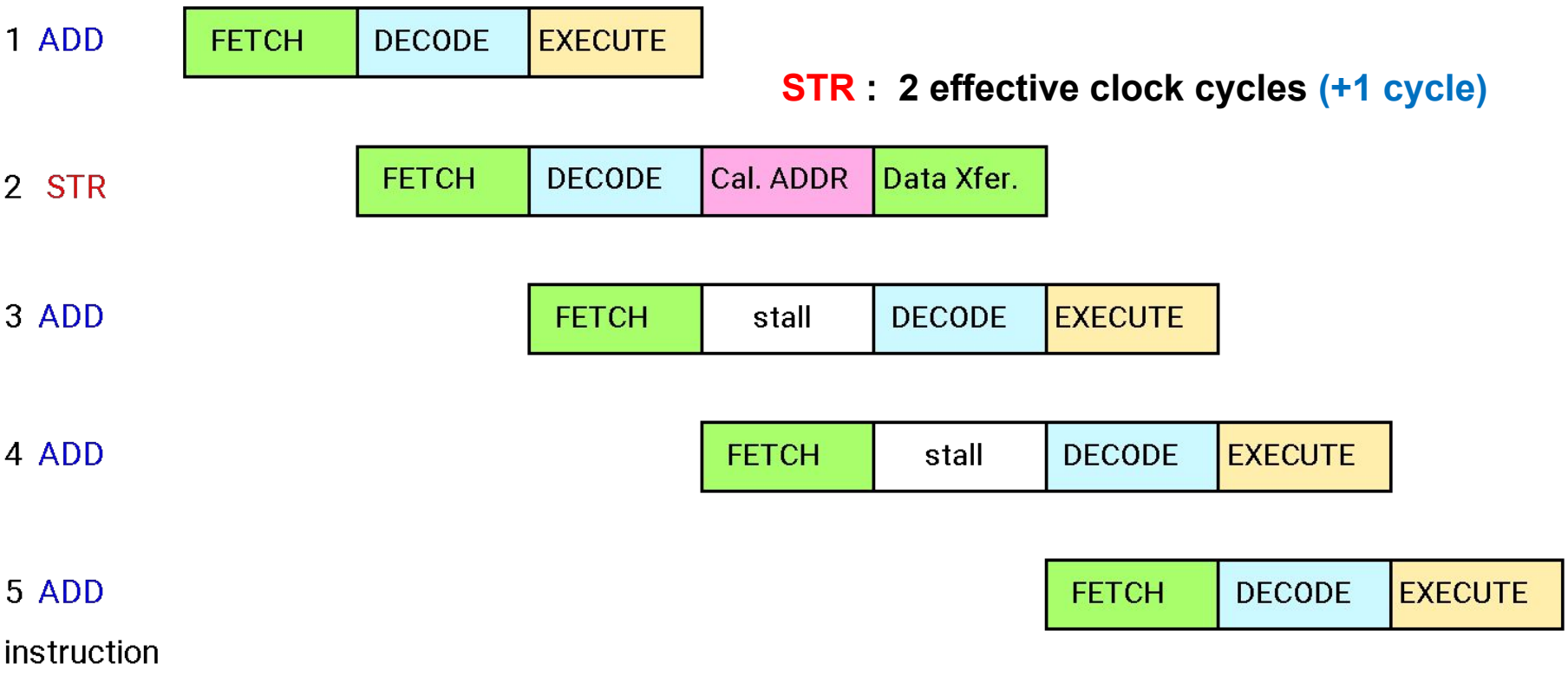
ARM7TDMI Pipelining



- **Simple instructions (like ADD) Complete at a rate of one per cycle**

ARM7TDMI Pipelining

- **More complex instructions:**



- Memory access (fetch, data transfer) in every cycle
- Datapath used in every cycle (execute, address calculation, data transfer)
- Decode logic generates the control signals for the data path used in next cycle (decode, address calculation)

ARM 3-stage vs 5-stage pipelining

* **Core architecture**

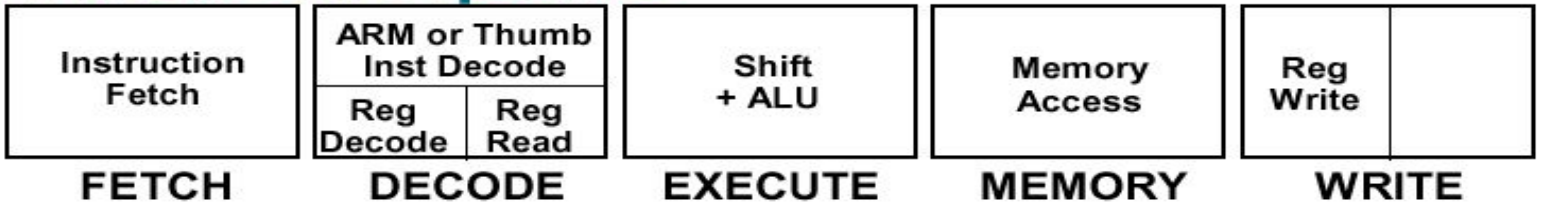
	ARM7	ARM9
Architecture	Von Neumann	Harvard
Pipelining	3-stage	5-stage
ISA	v4T	v4T

* **Pipeline stage**

ARM7TDMI Pipeline



ARM9TDMI Pipeline



‰ Fetch

– The instruction is fetched from memory and placed in the instruction pipeline

‰ Decode

– The instruction is decoded and register operands read from the register files. There are 3 operand read ports in the register file so most ARM instructions can source all their operands in one cycle

‰ Execute

– An operand is shifted and the ALU result generated. If the instruction is a load or store, the memory address is computed in the ALU.

‰ Buffer/Data

– Data memory is accessed if required. Otherwise the ALU result is simply buffered for one cycle

‰ Write back

– The result generated by the instruction are written back to the register file, including any data loaded from memory

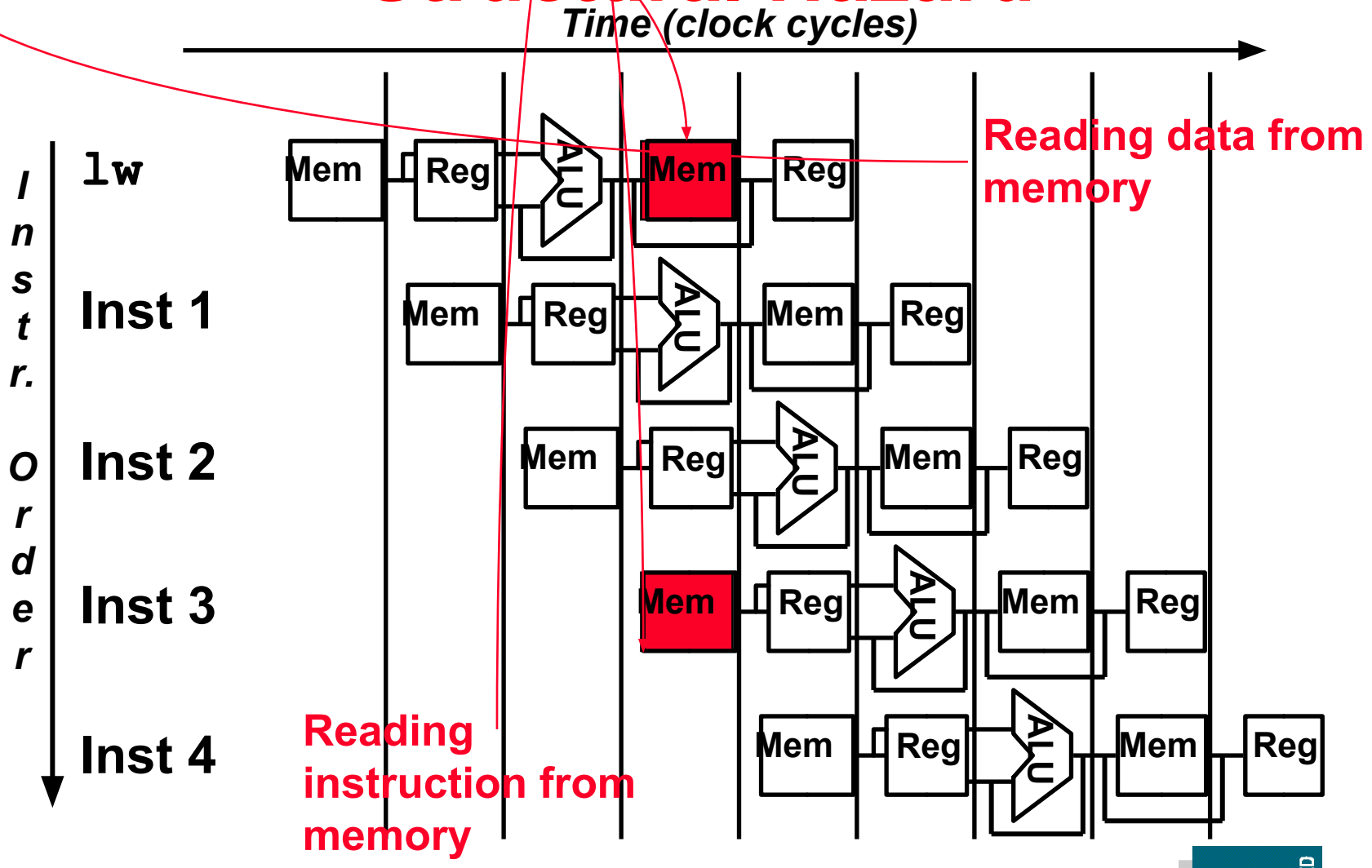
Pipeline Hazards

* **Pipeline Hazards**
  - structural hazards: attempt to use the same resource by two different instructions at the same time
  - data hazards: attempt to use data before it is ready
    - An instruction's source operand(s) are produced by a prior instruction still in the pipeline
  - control hazards: attempt to make a decision about program control flow before the condition has been evaluated and the new PC target address calculated
    - branch instructions

- **Can always resolve hazards by waiting**
  - **pipeline control must detect the hazard**
  - **and take action to resolve hazards**

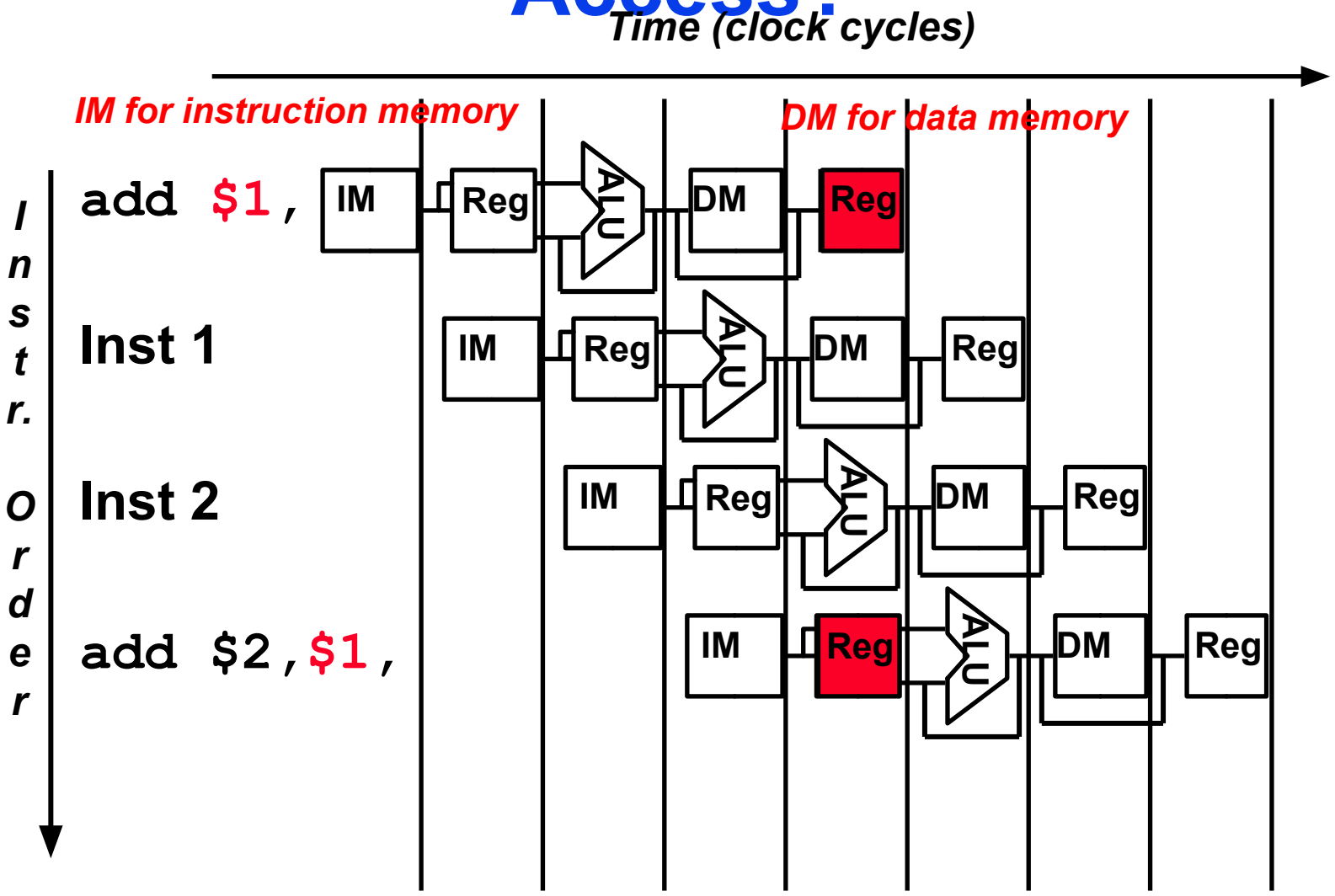
ARM POWERED ™

A Single Memory Would Be a Structural Hazard

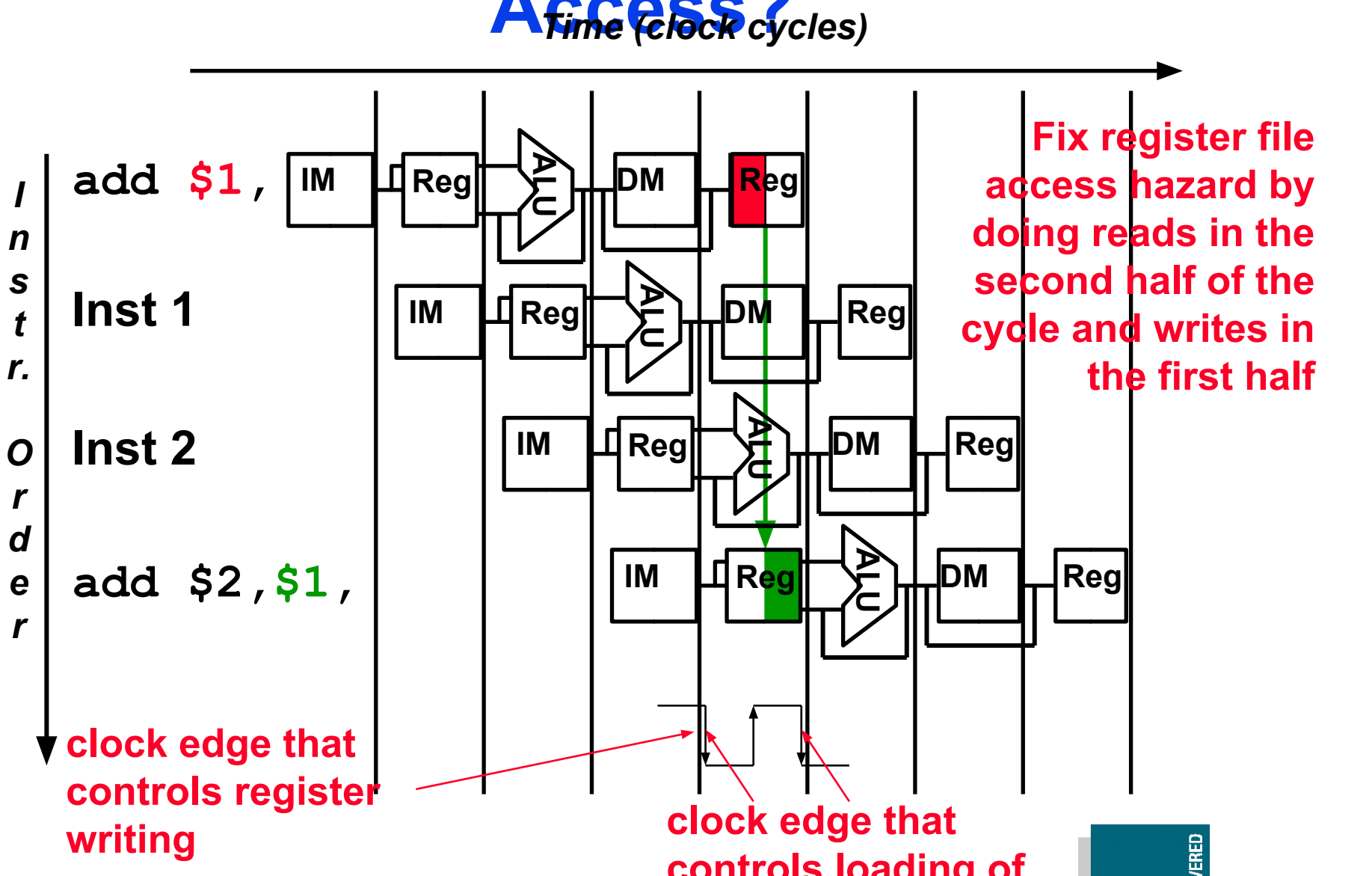


- Fix with separate instr and data memories (I$ and D$)

How About Register File Access?



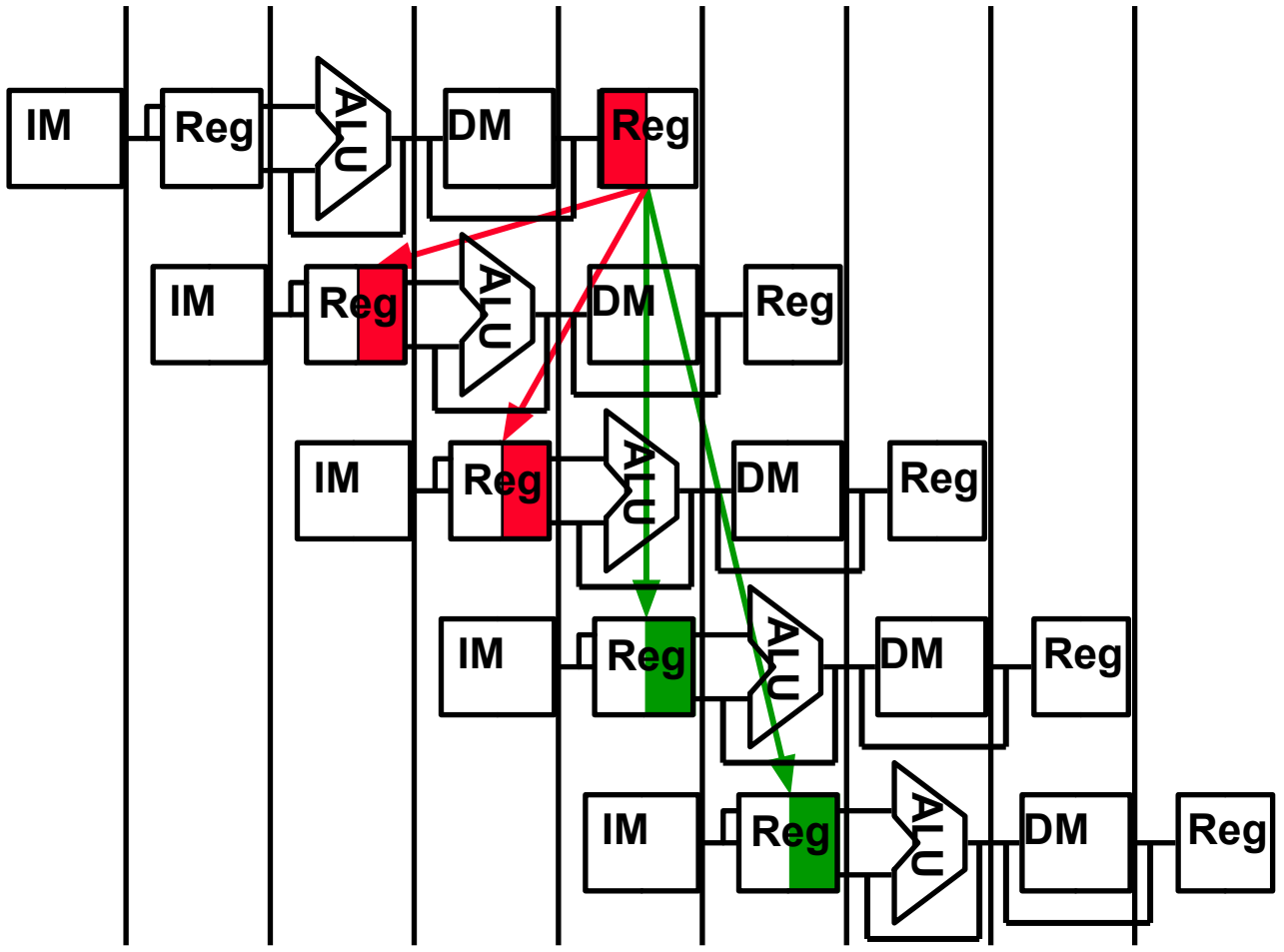
How About Register File Access?



Register Usage Can Cause Data Hazards

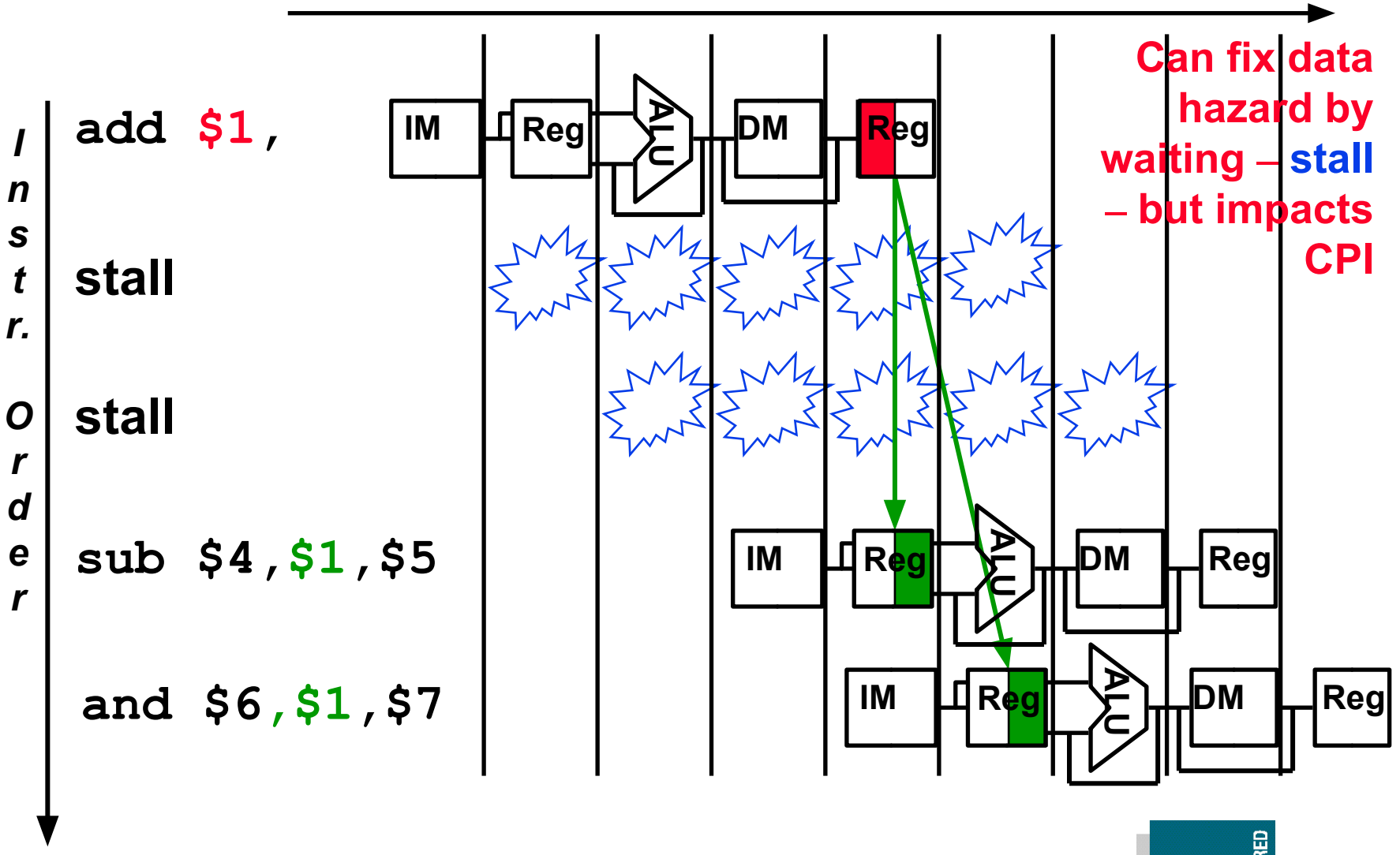
* Dependencies backward in time cause hazards

```
add $1,
sub $4,$1,$5
and $6,$1,$7
or  $8,$1,$9
xor $4,$1,$5
```



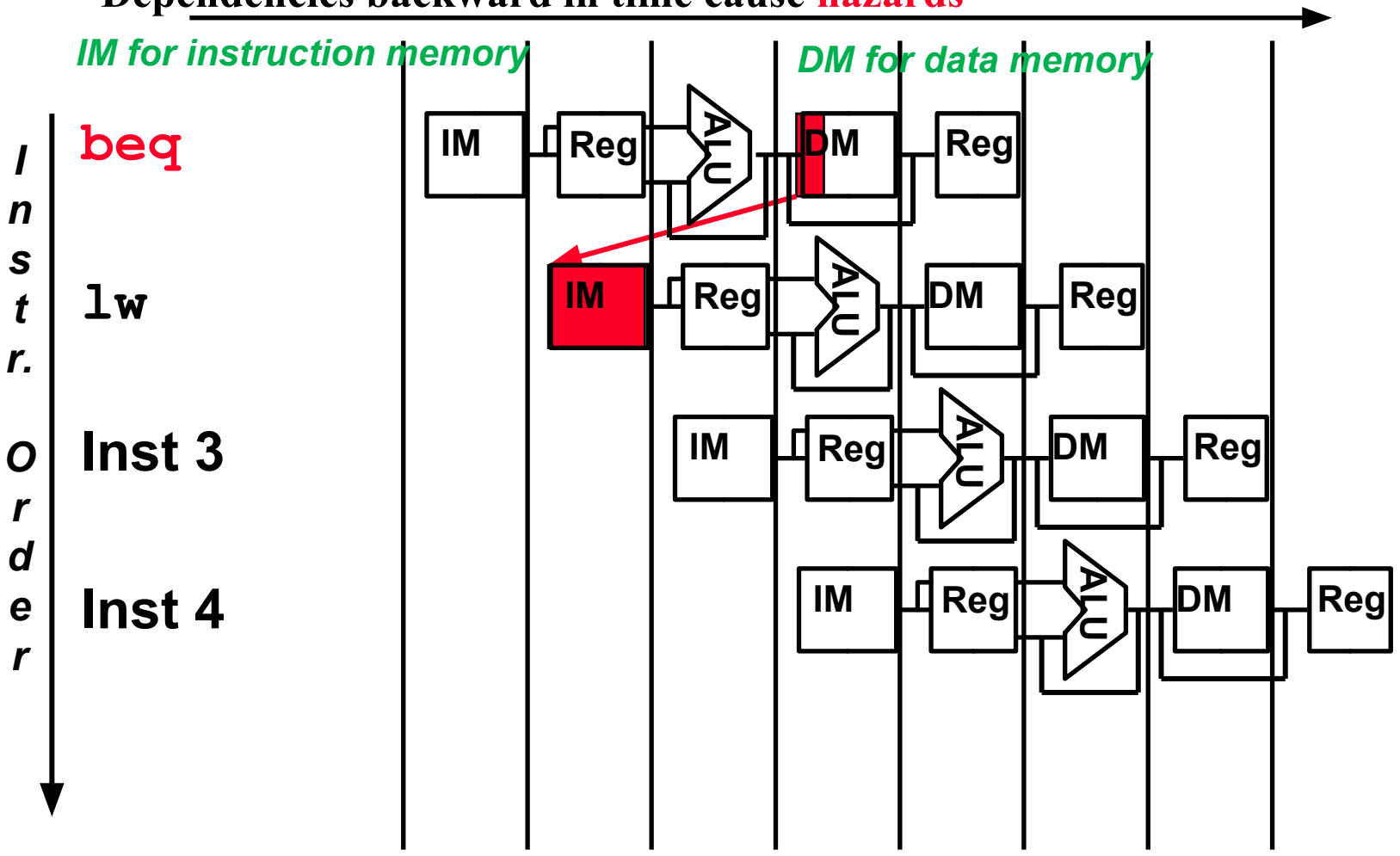
- Read before write data hazard

One Way to "Fix" a Data Hazard

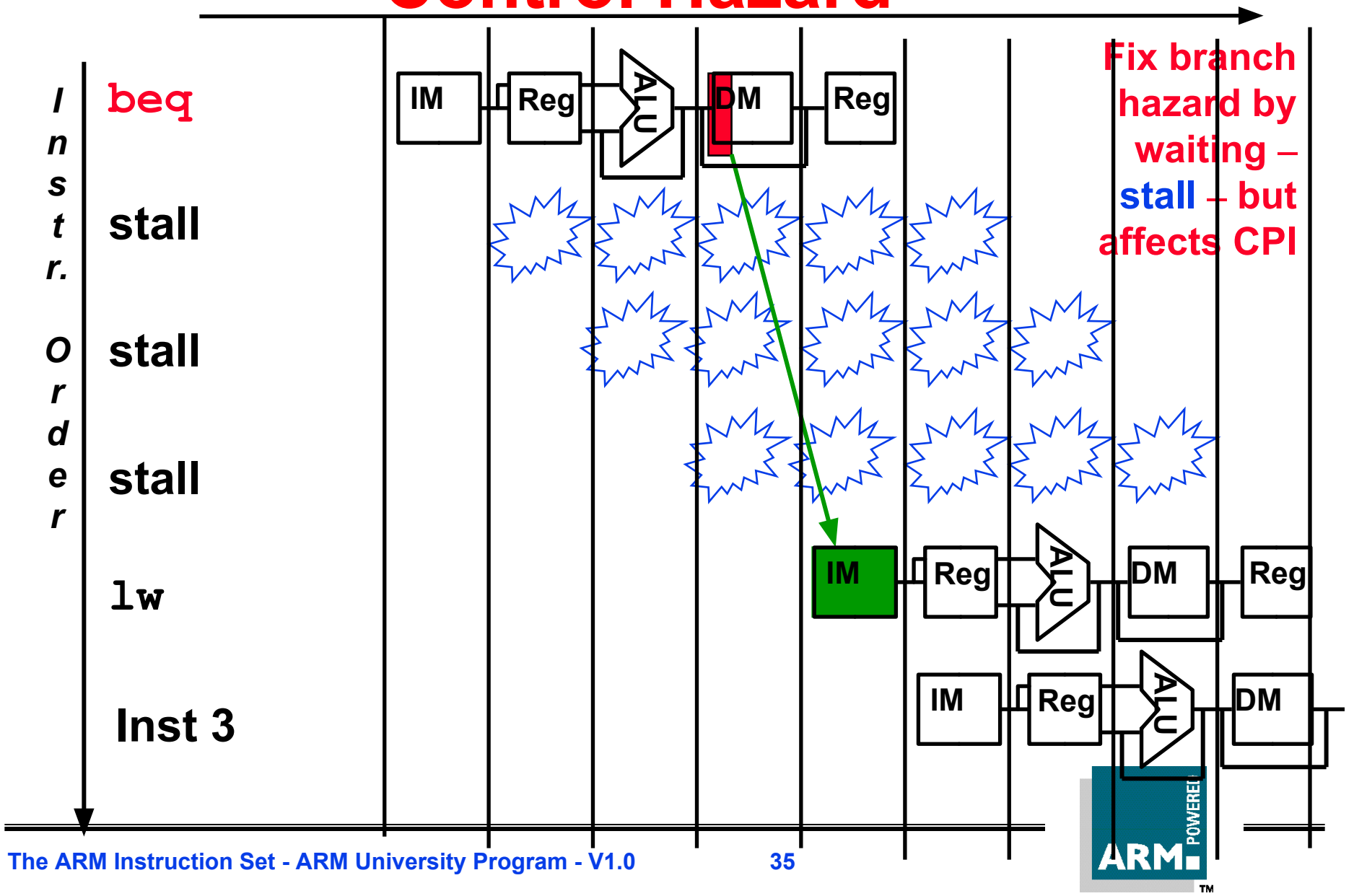


Branch Instructions Cause Control Hazards

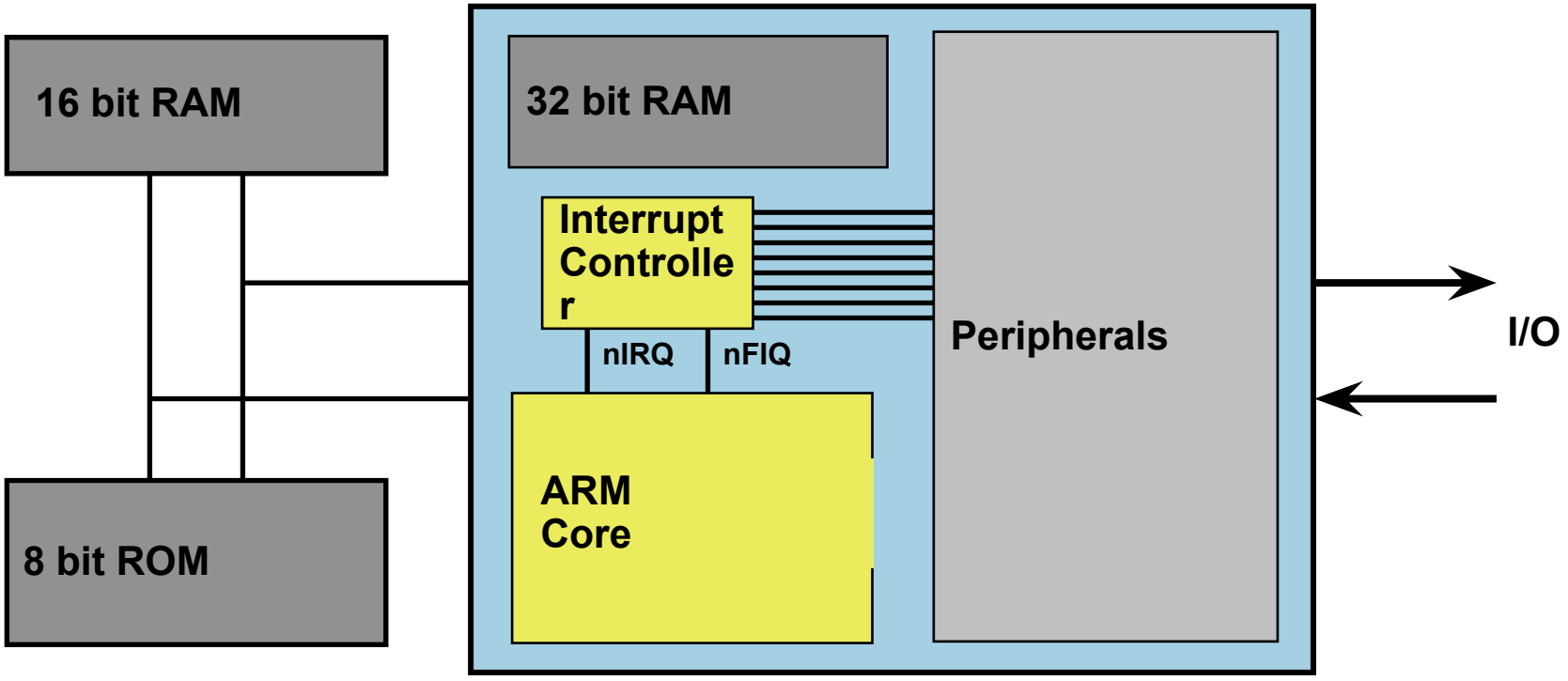
* Dependencies backward in time cause hazards



One Way to "Fix" a Control Hazard

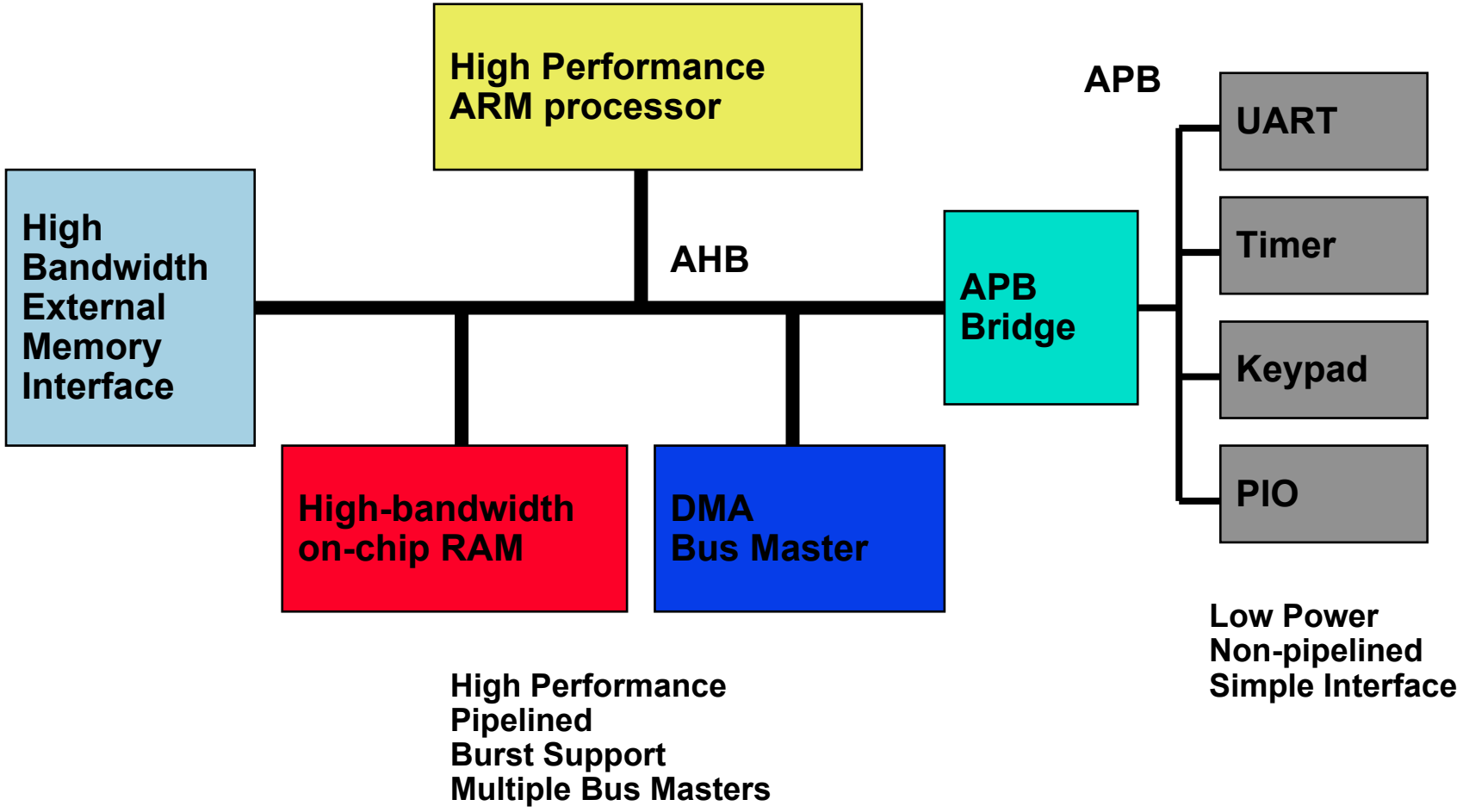


Example ARM-based System



An Example AMBA System

AMBA: Advanced Microcontroller Bus Architecture



Advanced Peripheral Bus (APB), Advanced High-performance Bus (AHB)

AMBA systems are based around two buses, a high performance system bus and a lower performance peripheral bus.

The high performance bus (AHB) should connect all of the high performance, high bandwidth modules, such as the ARM Processor, any DMA engines, perhaps some fast, 32 bit wide local RAM (or wider to suit the ARM Bus Master being used), an external memory interface, and the interface to the lower performance bus. The number of modules connected here should be kept to a minimum to reduce the bus loading on this high performance bus and allow it to run much faster.

The bulk of the design modules are placed on the lower performance Peripheral Bus (APB). Modules placed here are not accessed as frequently as AHB modules, and as a result of the APB timing, need not consume power (in their APB interfaces) when there is no APB activity. The address and data bus widths on the APB only need to be as wide as required, compared to the AHB where the maximum address and data widths are specified to maximize system performance in this critical area.

So when a design is being partitioned, the main design criteria are

AHB	APB
High Performance	Low Power
Frequent Access	Simple Interface
Pipelined timing	Non-pipelined

We don't go into multi-layer AHB, AHB Lite or any other AMBA configurations in this presentation.